

GRUPOJAP · DESAFIO TÉCNICO

Gestão de aluguer de veículos

Especificação funcional, modelo inicial de domínio e plano de implementação.

ASP.NET Core MVC · C# · SQL Server · Entity Framework Core

Documento de trabalho · 1 de outubro de 2026

1. Objetivo e âmbito

A aplicação permite gerir veículos, clientes e contratos de aluguer. Esta especificação traduz o desafio técnico fornecido em requisitos verificáveis e regista as decisões tomadas para iniciar o domínio.

Estado atual: o projeto ASP.NET Core MVC, as entidades, o `ApplicationDbContext` e as migrations iniciais estão preparados. A base `GrupoJapRentals` foi criada na instância local SQL Server Express e as migrations foram aplicadas; os controladores de gestão e formulários ainda serão implementados.

2. Stack e organização

- ASP.NET Core MVC em C# para rotas, controllers e views Razor.
- HTML, CSS e JavaScript para a interface; validação do servidor é sempre a autoridade.
- SQL Server com Entity Framework Core Code-First.
- Testes unitários com xUnit para as regras de domínio.

Organização inicial: `Models/` contém as entidades e enumerações; `Controllers/` contém os fluxos MVC; `Views/` contém as páginas Razor; `wwwroot/` contém CSS e JavaScript. A configuração EF Core e os serviços de negócio serão acrescentados nas fases seguintes.

3. Modelo de domínio



Cada contrato referencia exatamente um cliente e um veículo. Um cliente ou veículo pode estar associado a vários contratos ao longo do tempo. Os identificadores estrangeiros serão obrigatórios e as relações configuradas no `DbContext`.

Vehicle · veículo

Campo	Regra
Id	Chave primária gerada pela base de dados.
Brand , Model	Obrigatórios; máximo de 80 caracteres.
Registration	Obrigatória; máximo de 15 caracteres; índice único no SQL Server.
YearOfManufacture	Obrigatório; positivo e não posterior ao ano atual.
FuelType	Obrigatório; enum: gasolina, gasóleo, híbrido, elétrico, GPL ou outro. O valor por seleccionar é rejeitado.

Customer · cliente

Campo	Regra
Id	Chave primária gerada pela base de dados.
FullName	Obrigatório; máximo de 150 caracteres.
Email	Obrigatório; formato válido; máximo de 254 caracteres; índice único no SQL Server.
Phone	Obrigatório; por decisão inicial, exatamente 9 dígitos, sem espaços ou símbolos (formato nacional português).
DrivingLicenseNumber	Obrigatória; máximo de 30 caracteres.

4. Contratos e regras de negócio

RentalContract · contrato de aluguer

Campo	Regra
Id	Chave primária gerada pela base de dados.
CustomerId , VehicleId	Obrigatórios; têm de referenciar registos existentes.
StartDate	Obrigatória; não pode ser anterior à data atual.
EndDate	Obrigatória; tem de ser posterior à data de início.
InitialMileage	Obrigatória; quilometragem inteira igual ou superior a zero.

Prevenção de conflitos

As datas do contrato são datas civis inclusivas, sem horas. Para o mesmo veículo, dois contratos entram em conflito quando `inicioExistente <= fimNovo` e `fimExistente >= inicioNovo`. Assim, a data de devolução continua reservada ao contrato atual; o contrato seguinte começa num dia posterior.

A entidade contém uma operação para comparar períodos. A consulta de conflitos e a gravação atómica serão responsabilidade do serviço de aluguer, que deve impedir duplicações mesmo quando existam pedidos concorrentes.

Estado do veículo

O estado não é armazenado nem editável. Para uma data de consulta `hoje`, um veículo está alugado se existir um contrato para o qual `inicio <= hoje <= fim`; caso contrário está disponível. A disponibilidade futura também tem de ser verificada ao criar um contrato.

5. Integridade e validação

- Validação no servidor através de Data Annotations e validações entre campos.
- Índices únicos na base de dados para matrícula e email; a verificação da interface não substitui a restrição persistente.
- Normalizar espaços e capitalização da matrícula/email antes de comparar e gravar, mantendo a mesma regra em todos os fluxos.
- Validar sobreposição na camada de serviço e tratar violações de unicidade/conflito com mensagens compreensíveis.
- Não permitir eliminar clientes ou veículos referenciados por contratos sem uma decisão explícita sobre histórico.

Nota de fuso horário: a validação atual usa a data local do servidor. Ao definir o ambiente de execução, fixar o fuso horário de negócio para evitar diferenças junto à meia-noite.

6. Plano de implementação

1. **Persistência:** base `GrupoJapRentals` criada na instância SQL Server Express e migrations aplicadas; documentar a configuração para outros ambientes.
2. **Veículos e clientes:** implementar CRUD MVC, formulários, listagens, validações e tratamento de duplicados.
3. **Contratos:** implementar seleção de cliente/veículo, datas, quilometragem, conflito de períodos e cálculo de disponibilidade.
4. **Interface:** concluir navegação, mensagens de erro/sucesso, apresentação responsiva, acessibilidade e pesquisa nas listagens.
5. **Qualidade:** testar regras e fluxos, rever migrations, documentar setup e preparar a demonstração técnica.

7. Critérios de aceitação

- Não se consegue criar veículo sem os campos obrigatórios, com ano futuro ou matrícula duplicada.
- Não se consegue criar cliente sem os campos obrigatórios, com email inválido/duplicado ou telefone fora do formato acordado.
- Não se consegue criar contrato sem cliente/veículo, com início passado, fim inválido, quilometragem negativa ou período conflituoso.
- As listas apresentam o estado calculado do veículo com base nos contratos e na data consultada.
- As regras essenciais têm testes automatizados e a aplicação funciona com uma base SQL Server configurada.

8. Decisões a confirmar

- Confirmar se o telefone será sempre português (9 dígitos) ou se serão aceites indicativos internacionais.
- Confirmar se a disponibilidade apresentada deve ser apenas a atual ou também por data futura.
- Confirmar política de cancelamento/eliminação de contratos e manutenção de histórico.
- Escolher o servidor SQL Server e o fuso horário usados na demonstração.